

Exploring and evaluating linear regression methods for approximating the Runge function

Amund Rekkavik, Kaja Sandnes
Department of Physics, University of Oslo
(Dated: October 5, 2026)

Machine learning traces its origins to linear regression, some of the earliest and simplest mathematical optimization algorithms. Despite their simplicity, linear regression problems allow us to study the correct tuning of complexity, hyperparameters, and learning rates. We have studied linear regression methods for approximating Runge's function, famously unstable at high complexities, with the regression models: OLS, Ridge, and Lasso. These analytical estimators have also been compared with various gradient descent optimisers in terms of both error and computational cost. A 5-fold cross validated model selection was performed, and the optimal estimator for minimizing test error was found to be ordinary least squares at a complexity of polynomial power $p = 12$. Ridge was found to be best when imitating OLS at $p = 12$ with very low penalty parameter ($\lambda = 6.1 \times 10^{-7}$). Lasso struggled with convergence for minimizing parameters and placed last among the three models. Gradient descent converged to analytical solutions within 10^2 iterations and at a tolerance 10^{-4} , with the optimal optimizer being Adam.

I. INTRODUCTION

Polynomials are quite simple to work with. Anyone who has taken a course in introductory calculus can easily differentiate and integrate them. They behave predictably, and are easy to compute the values of through simple multiplication and addition. It is therefore quite obvious why we might want to deal with polynomials rather than other functions when we can. From the Weierstrass approximation theorem, we know that we can approximate any function with polynomials of increasing complexity. The higher the complexity, the better the approximation. But when attempting to actually compute these polynomials, problems can arise. When studying polynomial approximations of continuous functions, Carl Runge discovered that increasing polynomial degree will, for some functions, lead to oscillations at the endpoints of the interval we are approximating it on, leading to bigger and bigger errors. We consider the function $f : [-1, 1] \rightarrow \mathbb{R}$ given by:

$$f(x) = \frac{1}{1 + 25x^2} \quad (1)$$

Runge showed that increasing the degree of polynomial approximation leads to an unbounded increase in approximation error: [1]

$$\lim_{n \rightarrow \infty} \left(\sup_{x \in [-1, 1]} |f(x) - P_n(x)| \right) \rightarrow \infty$$

It should therefore be obvious that if one wants to approximate such a function, how one does it is of vital importance. We wish to explore and evaluate different linear regression models on tackling just this problem. Approximating Runge's function on the interval $[-1, 1]$.

We use three different models for linear regression: the ordinary least squares, ridge, and lasso. The ordinary least squares model works by minimizing the mean squared error of its training data [2]. We are, however,

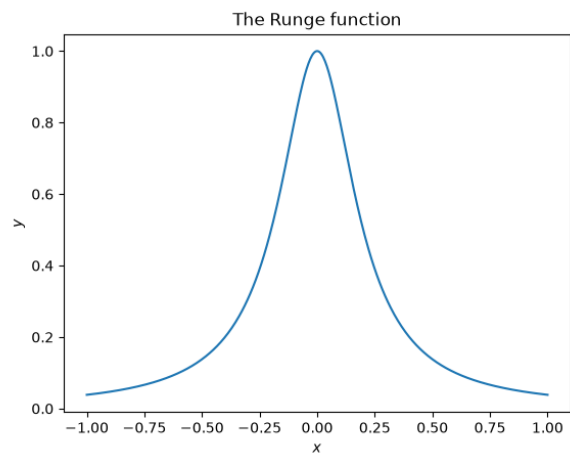


Figure 1. The graph of the function given by Eq. 1 on the interval $[-1, 1]$.

mainly interested in how a model performs on data outside its training set. A common problem for machine learning models, that also counts for our linear regression models, is that of *overfitting*. Increasing the complexity of a linear model beyond bound will, as we shall see, drastically increase its error through high variance. This is where ridge and lasso come in. These models penalize parameter size, and reduce degrees of freedom for models of mixed complexity artificially through the penalty parameter λ . We wish to study how well these models perform on the Runge function, and for what parameters they optimize performance.

For this example, our data is generated artificially as points on a grid and with some added gaussian noise. However, real data is rarely infinitely generable. When dealing with finite data, every data point is precious. In order to get the most out of a dataset, resampling methods are common. We will employ two of them. The

bootstrap, and k -fold cross validation. We will study how and why these methods of resampling may give different results for optimizing parameters.

We will also be evaluating different optimizer methods (gradient descent) that work without closed form solutions. Linear regression is such a nice sandbox for studying these optimizers because we already know what our answers are in the case of OLS and ridge. They are given by their closed form solutions. In addition, we know that we are optimizing a *convex* problem. Which guarantees us that when the gradient is zero, the cost function is minimized. We aim to study how the properties of our models impact the gradient descent methods and how they compare to one another. We will be utilizing standard gradient descent, different adaptive methods, and stochastic gradient descent.

In section II the theoretical groundwork for this paper will be revisited. We will present the different models and optimizers being used, and break down model error into bias and variance. Section III describes the implementation of these methods and the testing via resampling and comparison with closed form solutions. Section IV present the results from our work and discusses them. We will present results from the linear regression models and then the different optimizers, compare them, and discuss their relevance and reasonability. The final section: V concludes with our results.

II. BACKGROUND AND THEORY

A. The linear regression models

The problem of linear regression starts with a dataset of the independent variable $\mathbf{x}$ and corresponding values for the dependent variable (the target): $\mathbf{y}$.

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

For polynomial regression of degree p the coefficient parameter θ and the design matrix X are defined in the following way:

$$\theta = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{pmatrix}, \quad X = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^p \\ 1 & x_2 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^p \end{pmatrix}$$

A model's prediction: $\tilde{y}$ is given by:

$$\tilde{y} = X\theta \quad (2)$$

The goal of a model is to find the θ that minimizes its penalty on the training data. As mentioned in the introduction, we will be utilizing three different methods of linear regression, each categorized by its penalty:

- Ordinary least squares, which has a penalty given by the mean squared error: $\frac{1}{n} \|y - X\theta\|_2^2$.
- Ridge, which has a penalty that punishes the ℓ^2 -norm of the coefficient parameter: $\frac{1}{n} \|y - X\theta\|_2^2 + \frac{1}{n} \lambda \|\theta\|_2^2$.
- Lasso, which has a penalty similar to Ridge, but using the ℓ^1 -norm instead of the euclidian distance: $\frac{1}{2n} \|y - X\theta\|_2^2 + \alpha \|\theta\|_1$

Both OLS and ridge have differentiable penalties, which makes them nice to work with. The gradient can simply be derived analytically, set to zero, and thereby solved with respect to θ . Lasso is a little different as it uses the non-differentiable ℓ_1 -norm. For Lasso optimization, one must utilize another form of optimization such as the coordinate descent method used by the implementation in the scikitlearn library, or by using proximal gradient descent, presented later.

B. The resampling methods

We utilize two different resampling methods:

- Bootstrap, evaluates on N different datasets of same length as the original, composed of random samples from the original dataset drawn with replacement.
- k -fold cross validation, splits original dataset into k subsets. Trains on $k-1$ sets and evaluates on the one that is left out. Repeat k times so that every set has been an evaluation set once.

Utilizing these methods instead of only using a standard test-train-split allows us to get more use out of our data points. These methods also give us nice estimates for the uncertainty of our result, as we take the mean over bootstraps/folds. With enough samples, the distribution of the mean can be approximated as normal via the central limit theorem [3].

C. The gradient descent methods

For machine learning models where we cannot minimize the cost function by solving the equation $\nabla C(\theta) = 0$ analytically, we apply gradient descent methods that iteratively moves our parameters θ in the direction of the negative gradient of the cost function such that $C(\theta)$ is minimized stepwise. We will now present how the parameters fitted for the six gradient descent methods used in this project.

1. Plain gradient descent

Plain gradient descent simply subtracts a fraction of the gradient from the parameter vector. The parameter

vector at iteration $k + 1$ is then

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla C(\boldsymbol{\theta}_k) \quad (3)$$

The learning rate, η determines the size taken in the direction of the negative gradient.

2. Momentum

Momentum gradient descent include an exponentially weighted sum of all the gradients used so far so that more of the landscape formed by the cost function is captured. This way the parameter vector will converge faster. The momentum parameter γ controls the weight of the impact of the previous gradients.

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mathbf{v}_{k+1} \quad (4)$$

$$\mathbf{v}_{k+1} = \gamma \mathbf{v}_k + \eta \nabla C(\boldsymbol{\theta}_k) = \eta \sum_{j=0}^k \gamma^{k-j} \nabla C(\boldsymbol{\theta}_j) \quad (5)$$

3. AdaGrad

The AdaGrad method uses the squared gradient to adjust the step size for each component, so that the model will learn faster and slower in the directions where the gradient is respectively small and large.

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta}{\sqrt{\mathbf{r}_k} + \varepsilon} \nabla C(\boldsymbol{\theta}_k) \quad (6)$$

$$\mathbf{r}_k = \mathbf{r}_{k-1} + (\nabla C(\boldsymbol{\theta}_k))^2 \quad (7)$$

$\mathbf{r}_k$ represent the accumulated squared gradients which will monotonically increase, resulting in a constant decreasing step size. This is an advantage if the parameter vector is close to its optimum, however, if it is far from its optimum, the learning rate might become vanishingly small long before the minimum is reached.

4. RMSprop

Infinite memory is also a disadvantage if the landscape of the cost function changes so that the gradient of earlier iterations no longer reflect the current landscape. RMSprop does also scale the step size component wise, but uses an exponentially decaying average over the squared gradients instead of a cumulative sum

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta}{\sqrt{\mathbf{v}_k} + \varepsilon} \nabla C(\boldsymbol{\theta}_k) \quad (8)$$

$$\mathbf{v}_k = \rho \mathbf{v}_{k-1} + (1 - \rho)(\nabla C(\boldsymbol{\theta}_k))^2 \quad (9)$$

ρ controls how fast earlier gradients are forgotten.

5. Adam

While the momentum method uses an exponential moving average of the gradient and RMSprop uses the exponential moving average of the squared gradient, Adam combines them so that the step size is scaled component wise and includes the the most recent gradients when choosing which way to go. The momentum term $\mathbf{m}$ and the scaling term $\mathbf{v}$ are as follows

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \nabla C(\boldsymbol{\theta}_k) \quad (10)$$

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) (\nabla C(\boldsymbol{\theta}_k))^2 \quad (11)$$

Because $\mathbf{m}_k$ and $\mathbf{v}_k$ are biased by a factor $(1 - \beta^k)$ [4], the data must be multiplied with its inverse

$$\hat{\mathbf{m}}_k = \frac{\mathbf{m}_k}{1 - \beta_1^k}, \quad \hat{\mathbf{v}}_k = \frac{\mathbf{v}_k}{1 - \beta_2^k} \quad (12)$$

The final update becomes

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_k} + \varepsilon} \hat{\mathbf{m}}_k \quad (13)$$

6. Proximal gradient descent

Assume that the gradient of the cost function we want to minimize consists of a differentiable term f and non-differentiable term g . Then it can be shown that there exist an upper bound [5]

$$C(y) = f(y) + g(y) \leq g(y) + f(x) + \nabla f(x)^T (y - x) + \frac{\beta}{2} \|y - x\|_2^2 \quad (14)$$

where β is the curvature of a function that upper-bounds f at any point. The parameter y that minimizes this expression is given by

$$\text{Prox}_{g/\beta} \left(x - \frac{1}{\beta} \nabla f(x) \right) \quad (15)$$

Where the proximal operator is defined as

$$\text{Prox}_{\kappa}(x) := \underset{y \in \mathbb{R}^d}{\text{argmin}} \left\{ \kappa(y) - \frac{1}{2} \|y - x\|_2^2 \right\} \quad (16)$$

For proximal gradient descent, the parameter vector is updated by

$$\boldsymbol{\theta}_{k+1} = \text{Prox}_{\eta_k g}(\boldsymbol{\theta}_k - \eta_k \nabla f(\boldsymbol{\theta}_k)) \quad (17)$$

With Lasso optimization, the proximal operator becomes the soft threshold operator

$$\text{sign}(\boldsymbol{\theta}) \max\{|\boldsymbol{\theta}| - \alpha\} \quad (18)$$

D. The bias variance tradeoff

The bias variance tradeoff is the name given to the mathematical fact the the expected squared error ESE of a model can be broken down into the model's bias and the variance of its predictions. What follows is a derivation of this equality, and a discussion of how it changes when real data is used.

1. Derivation

To derive the bias-variance tradeoff, we start by assuming that our data is generated by:

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

We look at the expected squared error, or the ESE:

$$ESE = \mathbb{E}[(y - \tilde{y})^2] = \mathbb{E}[y^2] - 2\mathbb{E}[y\tilde{y}] + \mathbb{E}[\tilde{y}^2]$$

By the variance shortcut formula we have:

$$\mathbb{E}[\tilde{y}^2] = \text{Var}(\tilde{y}) + \mathbb{E}[\tilde{y}]^2$$

Substituting the expression for y :

$$\mathbb{E}[y^2] = \mathbb{E}[(f + \varepsilon)^2] = \mathbb{E}[f^2] + 2\mathbb{E}[f\varepsilon] + \mathbb{E}[\varepsilon^2]$$

Since f is non stochastic, and $\mathbb{E}[\varepsilon] = 0$ we get:

$$\mathbb{E}[y^2] = \mathbb{E}[f^2] + \sigma^2$$

Now looking at our next term:

$$\mathbb{E}[y\tilde{y}] = \mathbb{E}[(f + \varepsilon)\tilde{y}] = \mathbb{E}[f\tilde{y}] + \mathbb{E}[\varepsilon\tilde{y}] = \mathbb{E}[f\tilde{y}]$$

We are now left with:

$$ESE = \mathbb{E}[f^2] - 2\mathbb{E}[f\tilde{y}] + \mathbb{E}[\tilde{y}]^2 + \text{Var}(\tilde{y}) + \sigma^2$$

Factoring:

$$\mathbb{E}[(f - \mathbb{E}[\tilde{y}])^2] + \text{Var}(\tilde{y}) + \sigma^2$$

This is the form of our bias variance tradeoff:

$$ESE = \text{Bias}^2(\tilde{y}, f) + \text{Var}(\tilde{y}) + \sigma^2 \quad (19)$$

Calculating these statistics using our datapoints we arrive at another expression for the equation:

$$\frac{1}{n} \sum_i (f_i - \mu_{\tilde{y}})^2 + \frac{1}{n} \sum_i (\tilde{y}_i - \mu_{\tilde{y}})^2 + \sigma^2 \quad (20)$$

2. Explanation

The mean squared error here has three terms. The bias of the model, the variance of the model, and the inherent noise in the system. In the first term:

$$\frac{1}{n} \sum_i (f_i - \mu_{\tilde{y}})^2$$

The mean of $\tilde{y}$ runs over all values our model predicts. The mean of this expression runs over all values the target function takes on f_i , and sums their squared differences with our model mean. This is a measure of how far, on average, our model mean is from our target. This is our bias. The second term:

$$\frac{1}{n} \sum_i (\tilde{y}_i - \mu_{\tilde{y}})^2$$

This sample mean measures the mean squared distance from our model's predictions and their mean. This is our variance. The third term:

$$\sigma^2$$

Is simply the inherent variance in the data y_i . This noise is stochastic, and functions as a floor for the MSE.

3. What happens when true function values are unknown?

When we use our y values instead of the true f values when calculating the bias we include the ε in our calculation:

$$\mathbb{E}[(y - \mu_{\tilde{y}})^2] = \mathbb{E}[(f - \mu_{\tilde{y}} + \varepsilon)^2]$$

Expanding the squared parenthesis and applying the linearity of the expectation operator:

$$\mathbb{E}[(f - \mu_{\tilde{y}})^2] + 2\mathbb{E}[(f - \mu_{\tilde{y}})\varepsilon] + \mathbb{E}[\varepsilon^2]$$

Since $\mathbb{E}[\varepsilon] = 0$ and $\mathbb{E}[\varepsilon^2] = \sigma^2$ we get:

$$\mathbb{E}[(f - \mu_{\tilde{y}})^2] + \sigma^2$$

The σ^2 term is included in the bias, we can the simply write:

$$MSE = \text{Bias}^2(\tilde{y}, y) + \text{Var}(\tilde{y})$$

III. METHOD

A. Studying OLS and Ridge

The first step to studying approximations of the Runge function is studying the approximations of OLS and ridge at different complexities and hyperparameters. An equally spaced grid of $n = 100$ values in the range $[-1, 1]$ was generated, and function values for the Runge function corresponding to each point was calculated with an addition of gaussian noise of standard deviation $\sigma^2 = 0.01$:

$$x \in [-1, 1], \quad y = \frac{1}{1 + 25x^2} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

The data was then split into training and validation and scaled. The split is made so that we can validate our model on points it has not been able to optimize itself for. A standard validation size of 0.2 was used. Each column of the feature matrix was scaled to zero mean, and standard variance using the mean and variance of the training data in order to avoid data leakage. The target y was centered using the mean of the training data.

$$y' = y - \bar{y}, \quad X = \begin{pmatrix} \frac{x_0 - \bar{x}}{\sigma_x} & \frac{x_0^2 - \bar{x}^2}{\sigma_{x^2}} & \dots \\ \frac{x_1 - \bar{x}}{\sigma_x} & & \\ \vdots & & \ddots \end{pmatrix}$$

The scaling is not important for the OLS estimator, but when dealing with penalized estimators such as Ridge and Lasso, each feature must be penalized equally. If the means and variances vary drastically from feature to feature, the penalty parameter will punish disproportionately those with large numerical values. Proper scaling is even more critical when it comes to gradient descent, as the slopes of the gradient must be comparable. Another important thing to mention about the scaling is that we don't want Ridge and Lasso to punish the intercept of the model. The intercept of the OLS estimator is given as: [2]

$$\theta_0 = \bar{y} - \sum_{j=1}^{p-1} \bar{x}_j \theta_j \quad (21)$$

In our scaled space, we have: $\bar{y} = 0$ and $\bar{x}_j = 0$ which gives us an intercept of exactly 0. Thus we avoid having a bias column in our feature matrix at all.

After the generation and scaling of the data the OLS and Ridge models were fitted on the training data according to their closed form solutions:

$$\hat{\theta}_{OLS} = (X^T X)^{-1} X^T y \quad (22)$$

$$\hat{\theta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T y \quad (23)$$

And then validated against the validation set. Both the MSE - and the R^2 -scores were calculated. OLS was studied at different complexities, and Ridge was studied at both different complexities, and with different parameters λ .

1. Testing the code

To test the OLS code, an array consisting of two random integers in the interval $[0, 1]$ were generated. The OLS estimator was called on this data together with a unitary feature matrix:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

If the OLS code works as it should the composition:

$$\tilde{y} = X(X^T X)^{-1} X^T y = Iy = y$$

Reduces to the identity. The unit test asserts that these arrays are in fact equal.

To test the ridge code, we know that for $\lambda = 0$ it should return the exact same prediction as the OLS estimator. For the Runge data approximated with a degree 6 polynomial, it was asserted that the two differed by no more than a tolerance of $\epsilon = 10^{-9}$ in a test function.

B. Test and training error

From Hastie et al. we learn that when training a model on training data and calculating the MSE on both test data and training data, the training MSE should be monotonically decreasing with model complexity, the test MSE should be lower bounded by the training MSE , and the test MSE should follow a "U"-curve, provided the model complexity is allowed to grow large enough [3]. The figure Hastie et al. use to illustrate this phenomenon can be found in figure 2. We conducted an analysis on

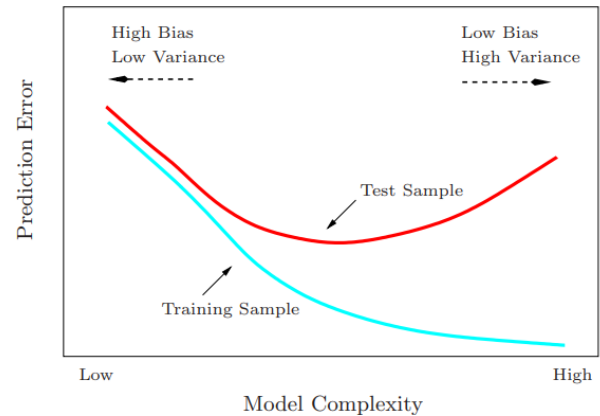


Figure 2. Figure from "The elements of statistical learning" by Hastie et al. [3]. Figure shows the evolution of testing and training error as model complexity grows.

both MSE s with an OLS estimator on the data from the Runge function. Like in the previous exercises, the Runge function was sampled at n equally spaced points in the interval $[-1, 1]$. The data was split and scaled with a standard training size of 0.2, and OLS models of differing complexities were fitted on the training data. The MSE was calculated on both the training, and the test data. For this analysis, we tried different values for model complexity and data points to see how they affect the overfitting of the models.

C. Resampling techniques

Two resampling techniques have been used in this analysis: bootstrapping and cross validation. Both methods are presented in the introduction of this paper.

1. Bias-variance analysis with bootstrapping

A bias-variance analysis of the OLS model on the Runge function was conducted with bootstrapping. The data was split into training and validation, with the standard validation size of 0.2. Then the training data was bootstrapped $k = 100$ times. Note that the data was rescaled inside every bootstrap to avoid data leakage. The model's predictions were stored in a large matrix and outside the bootstraps, the MSE, bias, and variance of the models was calculated over all bootstraps. This was done for model complexities up to a degree 15 polynomial.

The testing for this code was done by setting up an assert after the calculation of all mses, biases, and variances and asserting that the largest difference between the calculated MSE and the sum of the calculated bias and variance should be less than a tolerance $\varepsilon = 10^{-9}$. If this is correct, then the MSE has been correctly broken down into bias and variance.

2. k -fold cross validation

An analysis of both the OLS and ridge estimators was conducted using 5- and 10-fold cross validation. Same as for the bootstrapping method, data was scaled inside the different folds to avoid data leakage, and scores were calculated as means over the folds. Scores were calculated for OLS over model complexities ranging from degree 1 to 25. For ridge, scores were calculated from degree 12 to 22 and with $\lambda \in [10^{-8}, 10^2]$.

D. Studying gradient descent methods

We used the same data set as earlier to create a design matrix with maximum degree 5, and sent it through the same procedure of scaling, centring and splitting into training and test sets. To study the performance of the gradient descent methods, we use them to optimize OLS and Ridge because they have a comparable analytical solution. We started by defining the cost function of the Ridge model and its gradient

$$C(\theta) = \frac{1}{n} \|X\theta - y\|_2^2 + \frac{1}{n} \lambda \|\theta\|_2^2 \quad (24)$$

$$\nabla C(\theta) = \frac{2}{n} X^T (X\theta - y) + \frac{2}{n} \lambda \theta \quad (25)$$

When the penalty parameter λ is zero the cost function becomes that of OLS. We also made another function that computed the gradient numerically through automatic differentiation by using the `jax.grad()` function which takes the cost function and differentiates it with respect to its first variable, and wrapped it by `jax.jit()` which compiles the function on its first call, convenient for repeatedly use. The two gradients were compared by asserting that the maximum value of the absolute values of their difference was under a tolerance of 10^{-15} , which corresponds to machine precision, for θ_0 with normal distributed parameters and for different values of λ , as the test is being done every time a gradient descent method is running. We also made a test that printed the condition number of the matrix, and gave a warning if the learning rate $\eta > 2/\lambda_{max}$, which is the theoretical limit for divergence when using plain gradient descent.

We started by testing the plain gradient descent method, by passing the initial parameter vector θ_0 into a loop of a maximum of $k_{max} = 10000$ iterations. For each iteration, the gradient was computed by automatic differentiation and the parameter vector was updated by Eq. 3. The new parameter vector was compared to the analytical solution

$$\theta_{analytical} = (X^T X + \lambda I) X^T y \quad (26)$$

by storing the norm of their difference. At last the norm of the difference between the new and the old parameter vector was compared to a tolerance of 10^{-5} , which was the convergence criteria. Any difference smaller than this would stop the iteration. The procedure was repeated for different values of learning rate and penalty parameter, where the number of iterations was stored at each run. We used the stored differences to check at which iteration our calculated parameter vector converged to the analytical solution with a tolerance of 10^{-4} . We then plotted the number of iterations needed to converge as a function of the learning rate and penalty parameter.

Next step was to implement gradient descent methods for momentum, Adagrad, RMSprop and Adam. This was done by generalizing the iterating loop used for plain gradient descent so that the parameter vector could be updated by the desired gradient descent method. The equations for updating the parameter vectors for each gradient descent method are listed and explained in Section II C. Each method got their own function with their respective equations for updating the parameter vector, called within the iterating loop when updating the parameter vector. We then plotted all the methods together with iterations to convergence on the y -axis against penalty parameter on the x -axis for different initial values of the learning rate.

We wanted to fit the Lasso regression model, and started by defining the cost function and its derivative:

$$C_{Lasso}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \alpha \|\theta\|_1 \quad (27)$$

$$\nabla C_{Lasso}(\theta) = \frac{1}{n} X^T (X\theta - y) + \alpha \text{sign}(\theta) \quad (28)$$

Because the derivative of the term $\|\theta\|_1$ is not differentiable at $\theta = 0$, we instead use the sign-function as the derivative. We then used the gradient descent methods from earlier to solve the Lasso optimization problem, however the parameters of only three of the four gradient descent methods converged first at the convergence criteria 10^{-3} . By wrapping our cost function and using `jax.jit(jax.grad())` on the Lasso cost function, and then feeding it with zero, the gradient was ≈ 0.2 . However, `jax.grad(jnp.abs)(0.0) = 1.0`, which is not a valid sub-gradient, because the slope needs to lie under the graph $|\theta|$ which has slopes -1 and 1. We optimized Lasso again using the gradient descent methods with the automatic differentiated gradient, but the parameters did not converge either. We decided to write our own gradient descent method using the soft threshold operator as described in Section II C 6 because the operator is selective and set some parameter values to zero. First we defined the gradient of the smooth part of the cost function

$$\nabla C_{\text{smooth}} = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) \quad (29)$$

The update of the parameter vector is done according to Eq. 17, which is our case is

$$\theta_{k+1} = \text{Prox}_{\eta_k \alpha}(\theta_k - \eta_k \nabla C(\theta_k)) \quad (30)$$

$$= \text{sign}(\theta_k - \eta_k \nabla C(\theta_k)) \max\{|\theta_k - \eta_k \nabla C(\theta_k)| - \eta_k \alpha, 0\} \quad (31)$$

Scikit-Learn, which minimizes the same cost function C_{Lasso} , uses coordinate gradient descent to minimize the parameters, which we use as the exact parameters to compare our gradient descent parameters to. Then we plotted iterations to converge against different values of the penalty parameter, for a chosen initial learning rate.

At last we implemented a routine for stochastic gradient descent. We divided our data into m minibatches of size M , both the design matrix and the target data. Then we looped through n_{epochs} number of epochs. For each epoch, we iterate the data through a new loop m times which within a random number k between zero and m was generated, the learning rate $\eta = t_0/(t + t_1)$ is calculated for the hyperparameters $t_0 = 1$ and $t_1 = 10$ where $t = \text{epoch} * m + i$ and i is the current iteration, the gradient was calculated through automatic differentiation, using only the data from mini batch nr. k , and so the parameter vector was updated by one of the gradient descent methods. We plotted the difference between the parameters obtained and the analytical parameters from Eq. 26 for the gradient descent methods Momentum, AdaGrad, RMSprop and Adam against minibatch sizes from 1 – 25 for 500 epochs, and against epochs in range 10 – 2500 for $M = 5$.

E. Final analysis

The final analysis of the project is concerned with the model selection. Which model with what parameters op-

timizes the test error for the dataset? A 5-fold cross validation was performed on the three models: OLS, ridge, and lasso over different parameters (p, λ). Final reported values for the test error are calculated as the mean over the mean squared errors of the k folds:

$$\overline{MSE} = \frac{1}{k} \sum_i MSE_i \quad (32)$$

The standard error of the mean (reported as the uncertainty in the measurement of the MSE) is calculated in the following, standard way:

$$s = \sqrt{\frac{\frac{1}{k-1} \sum_i (MSE_i - \overline{MSE})^2}{k}} \quad (33)$$

The same program used in the k -fold cross validation earlier was reused for the purposes of the final analysis and model selection. There were changes made to the plotting, such as plotting heat maps instead of multiple curves for ridge and lasso, as more parameters have been swept in this analysis. All models have been tested with $p \in [1, 25]$ and $\lambda \in [10^{-10}, 10^2]$ in order to cover the full range of underfitting-overfitting and lambdas in the range of approximately OLS to overpenalising.

This approach works well for OLS and ridge, but since Lasso has no closed form solutions, bigger condition numbers mean the model struggles with convergence. Here, high polynomial degrees, and low values for λ mean big condition numbers. The maximum number of iterations for lasso was set to 10^5 . If the model did not converge, no MSE was calculated and the function returned a *nan* value.

IV. RESULTS AND DISCUSSION

A. Ordinary least squares

The MSE - and R^2 -scores for the regular OLS model fitted on the runge function were calculated at different complexities and can be found in figures 3 and 4: These figures show the classical "u-curve" one expects to see for increasing model complexity evaluated on validation data [2]. For low complexities the model lacks enough degrees of freedom to actually catch onto the model's patterns. For high complexities the model starts to fit into the specifics of the training set and starts missing the points in the validation set. From this initial test, the ideal complexity together with its corresponding MSE - and R^2 -score can be found in table I: This minimum actually

Degree	MSE	R^2
12	0.0061	0.9338

Table I. Ideal complexity and scores from first test of OLS on the runge data.

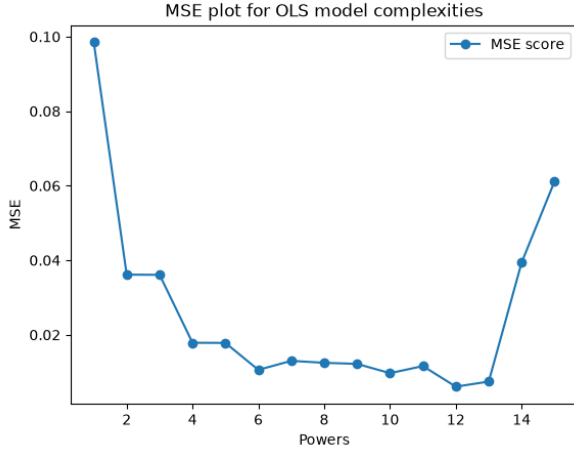


Figure 3. Plot of the mean squared error of the OLS model fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. Model complexity ranges from polynomial degree 1 to 15.

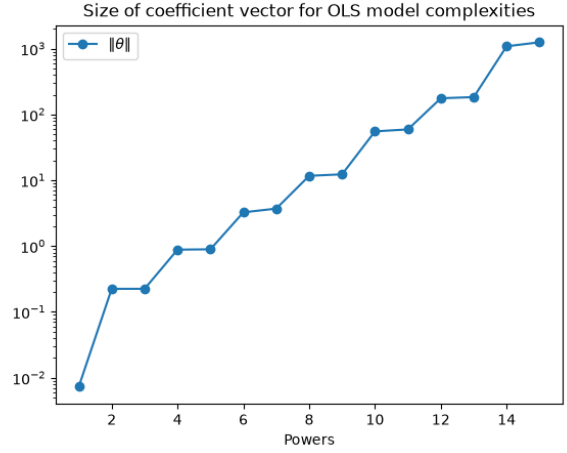


Figure 5. Plot of $\|\theta\|_2$ for the OLS model fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. Model complexity ranges from polynomial degree 1 to 15. Coefficient size is plotted on logarithmic scale.

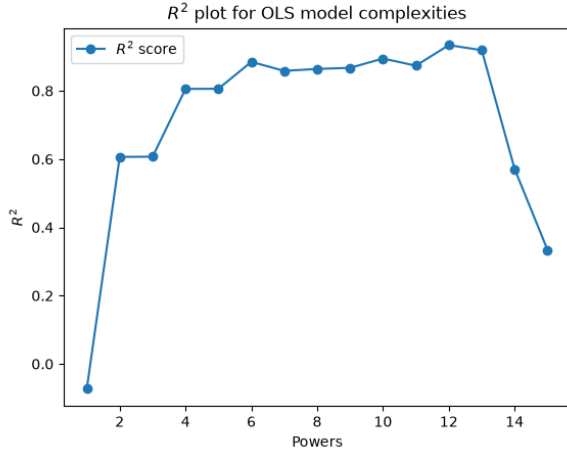


Figure 4. Plot of the R^2 -score of the OLS model fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. Model complexity ranges from polynomial degree 1 to 15.

beats the theoretical minimum for the MSE of σ^2 found in the bias variance tradeoff. This should, however, not be surprising, as there has only been one measurement done, and the deviation is not statistically significant. The sizes of the coefficient vectors were also calculated, and can be found plotted against model complexity in figure 5: The size of the coefficient seems to grow exponentially, evident by the almost linear curve on the logarithmic scale. This is classic evidence of overfitting. This trend is exactly what the ridge and lasso models attempt to fix.

B. Ridge

The purpose of ridge is to shrink large coefficients and fix ill conditioned matrices to hinder divisions by zero and overfitting for models of higher complexity [2]. We have thus calculated the MSE for ridge models for different values of the hyperparameter λ for different complexities, starting at polynomial degree 12, as that was the one found to be ideal from the previous test¹. In figure 6, precisely these curves have been plotted: Keep in mind

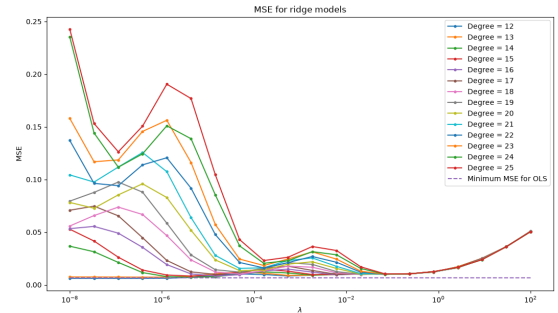


Figure 6. Plot of MSE for ridge models fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. Model complexity ranges from polynomial degree 15 to 25, and λ ranges from 10^{-8} to 10^2 . x -axis is on a logarithmic scale.

¹ If degree 12 is ideal for OLS, then ridge will improve the scores of higher complexities than 12. There is therefore no use in trying to see how ridge works for low complexities.

that for low values of λ , we are essentially seeing the OLS MSE . We know that ridge shrinks singular value modes by: [2]

$$\frac{\sigma_j^2}{\sigma_j^2 + \lambda}$$

In figure 5, we saw that as OLS started overfitting, the size of its parameter grew exponentially. Here we can clearly see that ridge is stopping the overfitting of complex models by killing large unwanted coefficients. In the large λ regime, we see that all models eventually converge towards the same MSE as $\|\theta\| \rightarrow 0$. The same tests we did for OLS have also been done for a ridge model of degree 25 so we can clearly see how increasing λ deals with an overfitting complex model. The MSE -score, R^2 -score and the size of the parameter can be seen plotted against λ in figures 7, 8, and 9:

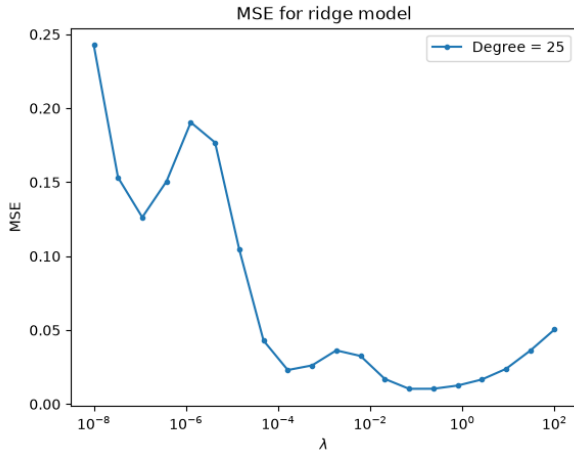


Figure 7. Plot of MSE for ridge model of polynomial degree 25 fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. λ ranges from 10^{-8} to 10^2 . x -axis is on a logarithmic scale.

Here we can clearly see that the model is drastically overfitting in the small λ , approximately OLS regime. Increasing λ quickly shrinks the size of θ , and cuts the MSE .

C. Test and training error

The data from the first comparison between test and training error can be found in figure 10. In the figure we clearly see the same trend as described by Hastie et al. and showed in figure 2. Here we have also plotted the noise floor $\sigma^2 = 0.01$. We can see that when the training MSE crosses the noise floor, the model starts overfitting, and the test MSE starts increasing. This makes sense, as we know from our derivation of the bias variance tradeoff. If the training MSE crosses the line

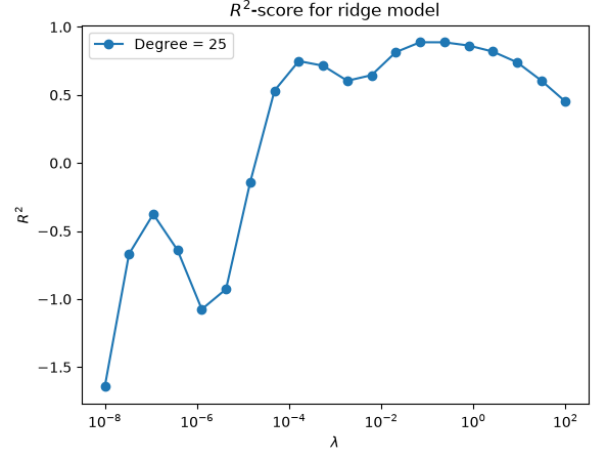


Figure 8. Plot of R^2 -score for ridge model of polynomial degree 25 fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. λ ranges from 10^{-8} to 10^2 . x -axis is on a logarithmic scale.

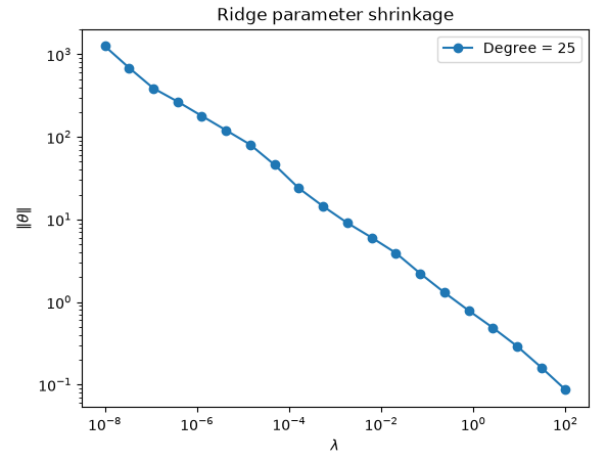


Figure 9. Plot of $\|\theta\|_2$ for ridge model of polynomial degree 25 fitted on data from the Runge function sampled at $n = 100$ points with noise of variance $\sigma^2 = 0.01$. λ ranges from 10^{-8} to 10^2 . Both axes are on a logarithmic scale.

σ^2 , then the model has gained enough degrees of freedom to actually fit onto the random noise in the training set. When this happens, it will start to miss points in the test set more frequently. This is exactly what overfitting is [3].

But how does the number of points in our dataset impact this trend? In figure 11 we have plotted test and training error against model complexity, same as figure 10, except this time the dataset consists of $n = 150$ points, and model complexities range up to polynomial degree 27.

This time, the model complexity needed for overfitting to begin is larger than in the previous case. We see that the number of points in the dataset is roughly propor-

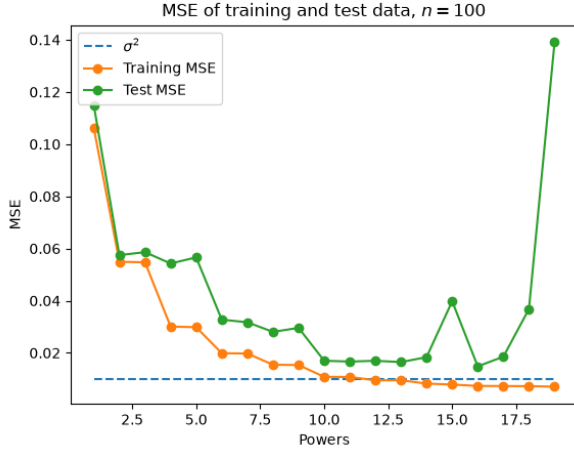


Figure 10. Plot of testing and training error as function of model complexity. Model is an OLS estimator trained on data from the Runge function with added noise of variance $\sigma^2 = 0.01$, with $n = 100$ equally spaced points in the interval $[-1, 1]$. Polynomial powers lie in the interval $[1, 19]$

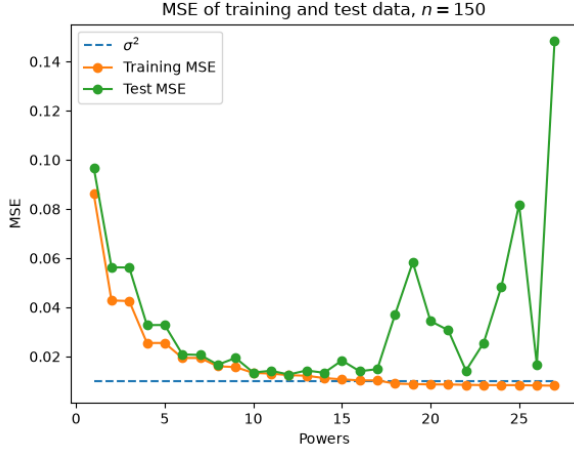


Figure 11. Plot of testing and training error as function of model complexity. Model is an OLS estimator trained on data from the Runge function with added noise of variance $\sigma^2 = 0.01$, with $n = 150$ equally spaced points in the interval $[-1, 1]$. Polynomial powers lie in the interval $[1, 27]$

tional to the model complexity for which the model starts overfitting. This makes sense, as the model needs more degrees of freedom to start fitting onto the random noise in a dataset consisting of more points. For the dataset of $n = 100$ points the overfitting starts roughly around degree 13. For the dataset of $n = 150$ the overfitting starts roughly around degree 18.

D. Bias-variance analysis with bootstrapping

The bias-variance analysis over $k = 100$ bootstraps was performed on the OLS estimator fitted on the Runge function. Figure 12 shows the mse broken down into bias and variance plotted over model complexities:

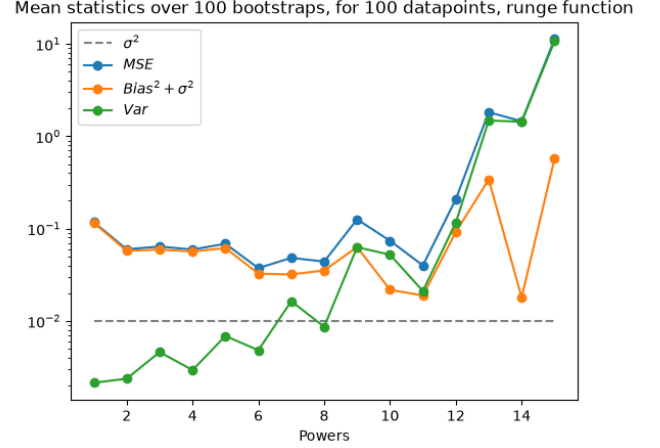


Figure 12. Bias-variance analysis performed on OLS estimator for polynomial powers in the range $[1, 15]$ on the Runge function with noise of variance $\sigma^2 = 0.01$ sampled at $n = 100$ equally spaced points in the range $[-1, 1]$. Analysis performed over $k = 100$ bootstraps.

As expected, we see low variance for low complexities, and increasing variance for higher complexities. Bias starts high as expected, decreases, but starts increasing again. We usually expect to see increasing variance, and decreasing bias with model complexity, where they are approximately equal around the *MSE* minimum [3]. But here, however, the bias rises beyond the *MSE* minimum. We performed an identical bias-variance analysis of the OLS model fitted on a regular third degree polynomial. Results can be found in figure 13. In this analysis we see that the bias does converge to its floor of σ^2 rather quickly, while the variance rises with model complexity. Just as one would expect. So what causes the bias to rise with model complexity in the first case? It must be the Runge function itself. Just as Runge predicted, higher polynomial powers lead to greater oscillations around the edges of the approximation [1]. This leads to an increasing bias, as well as an increasing variance for the polynomial fit, as we increase the complexity. From the bootstrap analysis we can find minimizing complexity, and its corresponding *MSE*. The results can be found in table II:

Degree	<i>MSE</i>
6	0.03544

Table II. Table of minimizing polynomial degree and minimized error for OLS over 100 bootstraps.

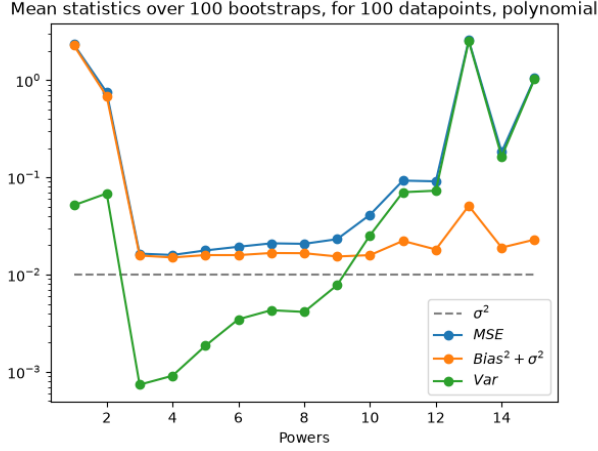


Figure 13. Bias-variance analysis performed on OLS estimator for polynomial powers in the range $[1, 15]$ on the polynomial function: $f(x) = 5x^3 + 3x^2 + 2x + 1$ with noise of variance $\sigma^2 = 0.01$ sampled at $n = 100$ equally spaced points in the range $[-1, 1]$. Analysis performed over $k = 100$ bootstraps.

E. k -fold cross validation

1. Ordinary least squares

In figures 14 and 15 you will find, respectively, 5- and 10-fold cross validated MSE for the OLS models over complexities:

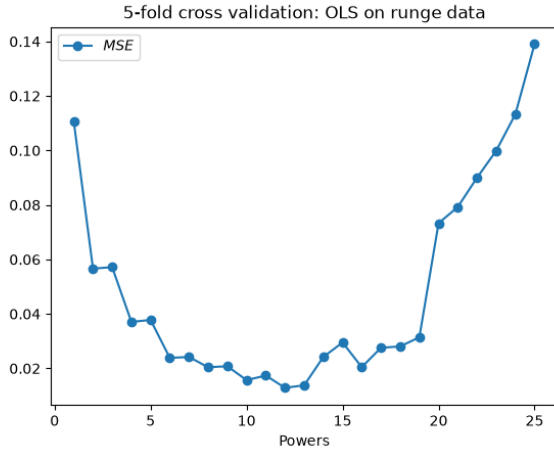


Figure 14. 5-fold cross validated MSE for OLS fitted on Runge function with noise of $\sigma^2 = 0.01$ with $n = 100$ plotted against model complexity. Polynomial powers lie in $[1, 25]$.

Both figures showcase the classic "U"-shape curve, but as we would expect from our previous results, the 10-fold cross validation (which essentially utilizes a training size of 0.9 rather than 0.8) is stable for higher complexities than the 5-fold. In table III their minimizing complexities

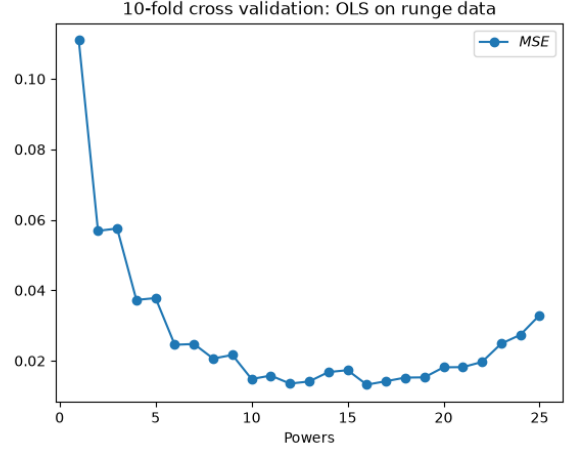


Figure 15. 10-fold cross validated MSE for OLS fitted on Runge function with noise of $\sigma^2 = 0.01$ with $n = 100$ plotted against model complexity. Polynomial powers lie in $[1, 25]$.

and minimized errors can be found. These results should

Fold	Power	MSE
5	12	0.0129
10	16	0.0132

Table III. Minimizing complexities and minimized test error for cross validated OLS.

not be very surprising. The bigger training size leads to a model that is more stable at higher complexities. The minimized MSE s do not significantly differ.

One interesting observation here is that the minimizing complexities found in the cross validation do not appear to line up with the minimizing complexity found from the bootstrap analysis. The bootstrap analysis also found significantly higher test errors. One possible explanation for this is that the way these methods resample vary quite drastically. In the bootstrap resample, the same data points can appear multiple times in one training set effectively limiting the size of the training sets. This should give our model lower stability at high complexities, which would explain the lower minimizing complexity.

2. Ridge

In figures 16 and 17 you will find, respectively, 5- and 10-fold cross validates MSE for Ridge models of different complexities over different values of the hyperparameter λ .

Here we see the same trend as previous ridge analyses. Ridge lowers the test error of more complex models for the correct λ . The higher the complexity, the higher λ needs to be to account for it. We see that in the 10-fold

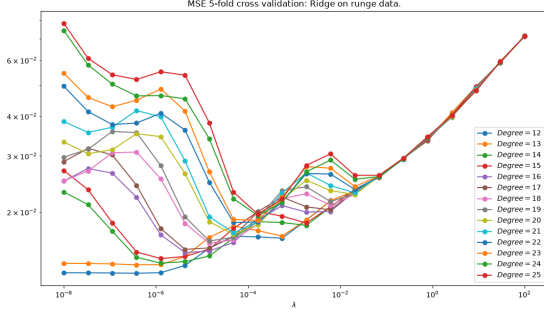


Figure 16. 5-fold cross validated MSE for ridge fitted on Runge function with noise of $\sigma^2 = 0.01$ with $n = 100$ plotted against model λ . Polynomial powers lie in $[12, 25]$ and λ lies in $[10^{-8}, 10^2]$.

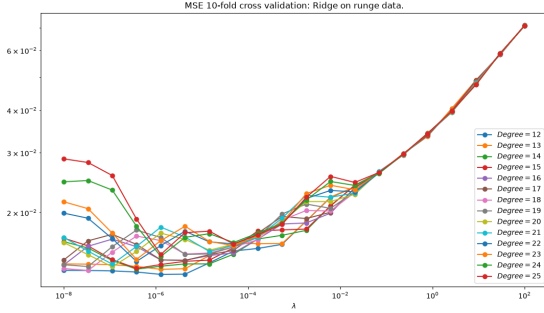


Figure 17. 10-fold cross validated MSE for ridge fitted on Runge function with noise of $\sigma^2 = 0.01$ with $n = 100$ plotted against model λ . Polynomial powers lie in $[12, 25]$ and λ lies in $[10^{-8}, 10^2]$.

cross validation, lower values for λ are preferred. This is in line with bigger training sets giving more stable models at higher complexities. Bigger training sets can do part of λ 's job, so its value must be lower here.

F. Gradient descent

We used automatic differentiation in order to obtain the gradient used in the gradient descent methods, which works by repeatedly applying the chain rule on the sequence of elementary arithmetic operations executed by the computer [4], costing only a fraction more than the original program. While symbolic differentiation manipulates the symbols of the function by which can be arbitrarily complex, and numerical differentiation handles numerical values at finite points, which can lead to rounding errors, automatic differentiation avoids both these problems and corresponds to the analytical gradient to machine precision. Also parameter it costs only a constant factor rather than one function evaluation [4]. It is therefore handy to use in our case because the gradient

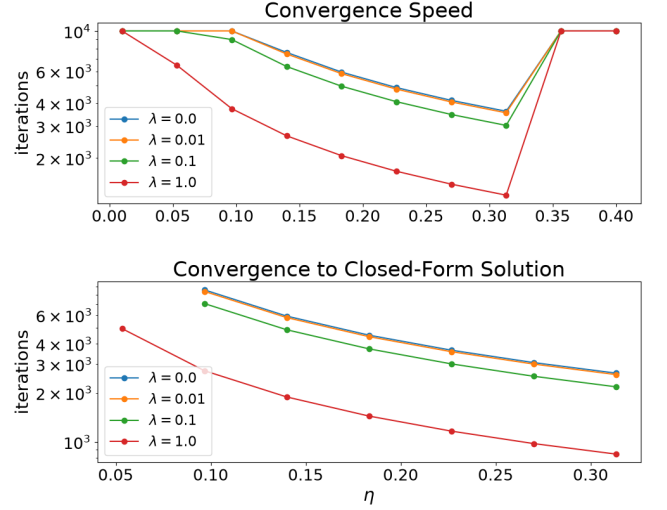


Figure 18. The number of iterations needed for the parameter vector to converge, and to converge to closed form solution on the y-axis and learning rates η on the x-axis for the plain gradient descent optimising Ridge (and OLS for $\lambda = 0$). Each dotted line represents a penalty value λ . Model is optimized with polynomials at maximum degree 5.

of the cost function is used many times.

We have used various gradient descent methods to optimize parameters to analytically solvable optimization problems. This way we can study how and if they converge and for which hyperparameters they perform best. Our method works if its parameters converge to a closed form solution that we know has low MSE. How many iterations it takes to get there, is also a measurement of its performance. We will now discuss the results for the Runge function from OLS, Ridge and Lasso regression with respect to the gradient descent methods used.

In Figure 3 we see that the number of iterations needed for convergence decrease significantly as the learning rate η increases. This is expected as η for plain gradient descent directly determine the step size. We see that for small values for η and λ the parameters does not converge at all, while the lines with higher penalty parameter λ tend to converge faster. This is a result of better conditioning of the Hessian $H = \frac{2}{n} \mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{I}$. Because the plain gradient descent learns equally fast in all directions, the step size depends on the gradient value, which is determined by the eigenvalues of the Hessian, as they determine the curvature of the cost function. Better conditioning of the Hessian decreases the difference in the largest and the smallest eigenvalue, so that the step size equalizes more in each direction, causing Ridge regression to converge faster. In the upper part of the figure we see that the lines diverges at $\eta \approx 0.33$. This is because the learning rate exceeds the theoretical limit for stability at $\eta > 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of the Hessian. This causes the component along the steepest direction to diverge, and is also a reason of why we want well conditioned Hessians. It is worth

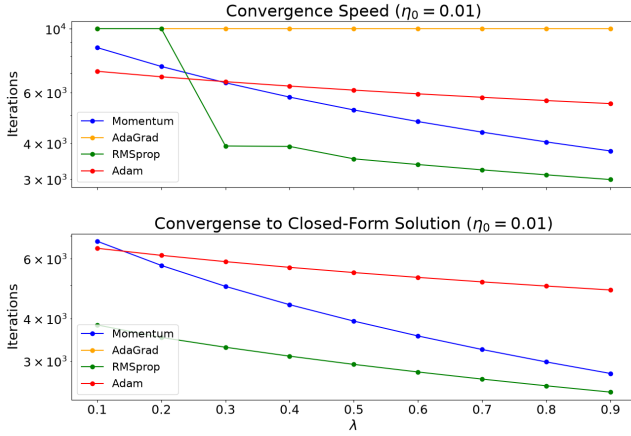


Figure 19. Iterations to convergence and convergence to closed-form solution are plotted against penalty parameters λ for the gradient descent methods momentum, AdaGrad, RMSprop and Adam for initial learning rate $\eta_0 = 0.01$

remembering that plain gradient descent performs better on Ridge optimization, as it converges faster due to more well conditioned Hessians. Another thing worth mentioning is that the design matrix has powers of maximum 5, even though the optimal power for OLS is 12, as seen in Figure 3. we see in the figure that the OLS-closed form solution is sufficiently good at degree 5 as well. Approximating polynomials of higher powers increases the condition number of the Hessian and slows the learning. Therefore one might need to use other models, such as Ridge, or other gradient descent methods. Figure 19 shows that AdaGrad does not converge at all, something that makes sense considering its accumulating sum of squared gradients making the step size decrease for each iteration, and the learning effectively stops before any minima is reached. RMSprop however that uses an exponentially decaying average over the squared gradients such that the step sizes are stable, converges fastest to closed-form solution for all penalty parameters for a small initial learning rate. We see that the momentum also performs well especially on high penalty parameters. Because the four methods changes their step size for each update of the parameter vector, the initial value of the learning rate η_0 is of importance when fitting the models. We see in Figure 20 that for a larger initial value $\eta = 1$, AdaGrad performs significantly better, while RMSprop performs significantly worse. It seems like RMSprop en Momentum performs well in regimes with small learning rates, while AdaGrad and Adam performs well in regimes with large learning rates.

In Figure 21 we see that none of the gradient descent methods Momentum, AdaGrad, RMSprop or Adam converged to the closed-form solution. When using automatic differentiation or the analytical gradient, the operator will never be selective and set some parameter values to zero, which is how the Lasso optimization is designed. We can see that our proximal gradient descent, which do

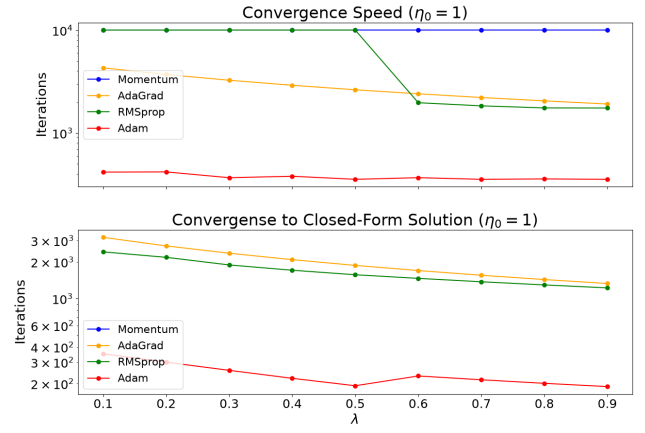


Figure 20. Iterations to convergence and convergence to closed-form solution are plotted against penalty parameters λ for the gradient descent methods momentum, AdaGrad, RMSprop and Adam for initial learning rate $\eta_0 = 0.01$

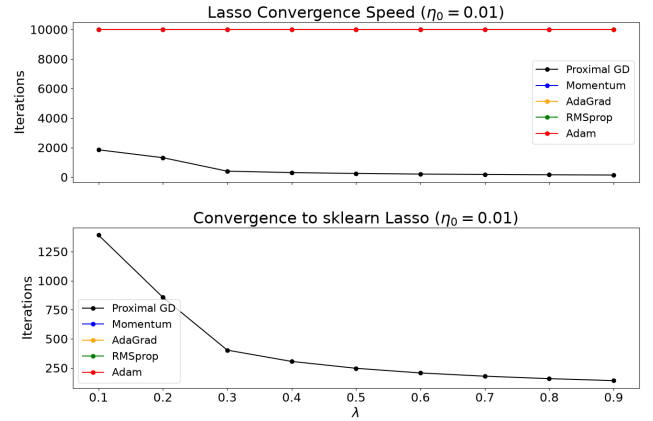


Figure 21. Iterations to convergence and convergence to Scikit-Learn's coordinate descent on y -axis plotted against penalty at the x -axis. The gradient descent methods Momentum, AdaGrad, RMSprop, Adam and Proximal GD was fitted on the Lasso model.

set parameter values to zero did converge to the closed-form solution.

In Figure 22 we see that by increasing the number of epochs, the accuracy in the estimation will increase. The most significant improvement in accuracy is by changing from 10 to 500 epochs. The accuracy as a function of mini-batch oscillates a lot for AdaGrad, RMSprop and Adam, but the trend seems to be that the accuracy decreases when the size of the minibatch increases. The accuracies are lower when using stochastic descent because the gradient is only calculated on a subset of the data. The oscillations can be explained by the gradient noise, which makes the parameter values jump around the minima, but rarely reach it. Momentum oscillates less because unlike the other methods, the gradient is not squared at each step. Because the gradient is noisy, it be-

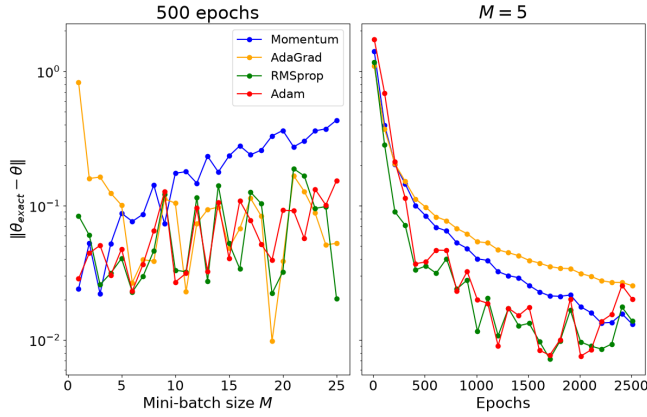


Figure 22. Accuracy in parameter optimization is plotted against mini-batch size M for 500 epochs and against epochs for $M = 5$ for the gradient descent models Momentum, AdaGrad, RMSprop and Adam.

comes important to choose the learning rate with care, it should be small enough to keep the noisy gradient under control, but not restricting it from jumping out of local minima. The upside with SGD is the decreased amount of computations for each gradient. For large datasets, that are complex and has many local minima, stochastic gradient descent are a good choice. However for our small dataset which does not tend to have many local minima, and does not demand a lot of computational power, the best choice would be to compute the full gradient, and trade computational cost against increased accuracy.

At lower learning rates, RMSprop reaches the closed form solution of OLS at $4 \cdot 10^3$ iterations. At a higher learning rate, Adam reaches the closed form solution at $4 \cdot 10^2$ iterations, which is ten times better. With respect to computation cost-accuracy trade, Adam is the best method to optimize OLS and Ridge. Which might not be surprising since the method uses per component scaling and has a memory of the earliest gradients. Proximal gradient descent with soft thresholding is the absolute best method to optimize the Lasso, since it sets parameters to zero.

G. Final analysis

Conducting a 5-fold cross-validated model selection based on optimized MSE with $n = 100$ and $\sigma^2 = 0.01$ yields the results found in table IV: Discuss briefly why automatic differentiation is neither symbolic nor numerical (finite-difference) differentiation, and what its cost is relative to one evaluation of the cost function.s

The model performances across different parameters can be found in figures 23, 24, and 25

The plots of the performances show us that both OLS and ridge find themselves at a stable minimum, with slowly increasing MSE in all directions. This tells us

Model	Degree p	λ	CV MSE
OLS	12	—	0.01288 ± 0.00123
Ridge	12	6.158×10^{-7}	0.01283 ± 0.00278
Lasso	8	4.833×10^{-5}	0.02005 ± 0.00425

Table IV. 5-fold cross validated minimized MSE for OLS, ridge, and lasso. Parameter sweep for $p \in [1, 25]$ and $\lambda \in [10^{-10}, 10^2]$. Data with $n = 100$, $\sigma^2 = 0.01$.

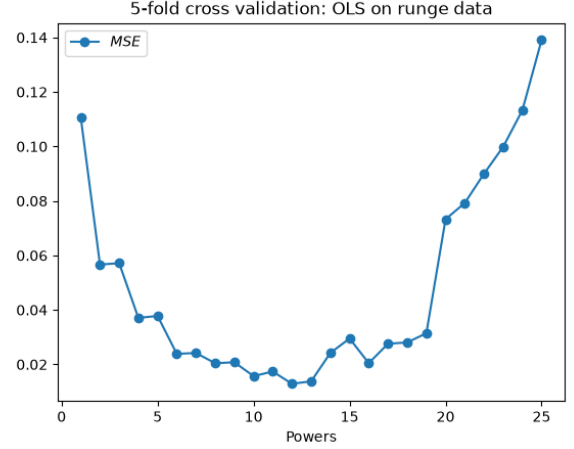


Figure 23. 5-fold cross validated MSE for OLS on Runge data with $n = 100$ and $\sigma^2 = 0.01$. This is the exact same result as from task d) showing the characteristic "U"-shaped curve that goes from underfitting to overfitting.

that the minimum value found is no mistake. It might vary slightly from power to power, and λ to λ , but this is the correct area to be in for minimal MSE . For lasso, the story is more unfortunate. The bottom right area of its heat map is gray, telling us that the model failed to

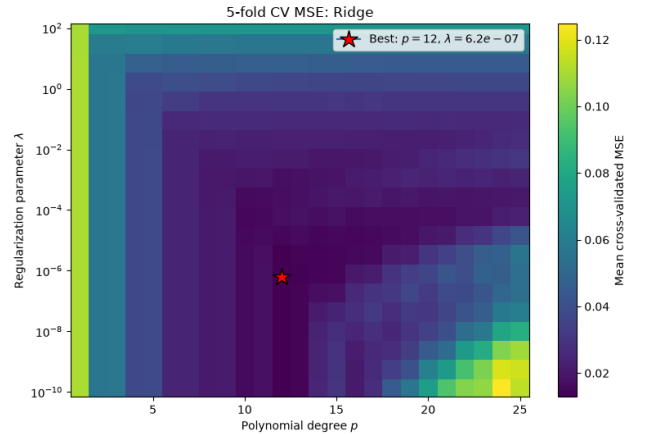


Figure 24. 5-fold cross validated MSE for ridge on Runge data with $n = 100$ and $\sigma^2 = 0.01$. The plot shows a heat map of the MSE across both polynomial degree and regularisation parameters

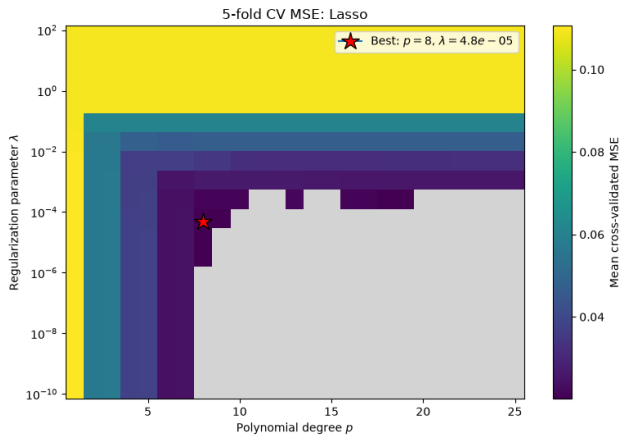


Figure 25. 5-fold cross validated MSE for lasso on Runge data with $n = 100$ and $\sigma^2 = 0.01$. The plot shows a heat map of the MSE across both polynomial degree and regularisation parameters. Values of (p, λ) where the model did not converge within 10^5 iterations are marked with gray. Keep in mind that sklearn’s definitions of the ridge and lasso penalties divide the ridge penalty by a factor of n , but not the lasso penalty. The λ values in these plots are therefore not directly comparable.

converge for high condition numbers.

From table IV we see that there are no statistically significant differences in performance between OLS and ridge. Ridge gets a very slightly lower MSE , but not outside our range of $\pm SEM$. Looking at its parameter values (same degree, and very low λ) it appears to be basically mimicking OLS. Lasso performs worse than the other two, but this is likely due to its failure to converge at high condition numbers. At degree 12 and a lower λ it should also be able to approach the MSE of OLS.

We know that increasing complexity kills bias at the expense of increased variance. Increasing λ kills variance at the expense of bias, as the ridge and lasso models are not unbiased like the OLS [2]. From figure 12 we see that the variance increases for high complexities, as one would expect. But the bias also increases as the approximations start oscillating around the edges of the Runge function. Adding additional bias at higher complexity via the ridge or lasso models as a way to decrease variance do therefore not seem like good ways if decreasing the MSE . This might be why both ridge and lasso are at their best imitating OLS.

V. CONCLUSION

We have used ordinary least squares (OLS), ridge, and lasso regression models to approximate the Runge function Eq. 1.

For the closed form solutions of OLS and ridge we have determined that (in line with the theory from Hastie et al. [3]) the test MSE of OLS forms a "U"-shaped curve when plotted against model complexity, as it goes

from underfitting to overfitting. Increasing training size makes larger complexities more stable as the model overfits more slowly, and so does switching to a ridge model with penalty parameter scaling along with the model complexity.

A 5-fold cross validated model selection was performed on OLS, ridge, and lasso. Fitting polynomials of complexities in the range $[1, 25]$ and with penalty parameters in the range $[10^{-10}, 10^2]$ on data sampled at $n = 100$ points in the range $[-1, 1]$ with added gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2 = 0.01)$. We have determined that OLS and ridge perform identically to statistical significance on the problem, reaching a minimum MSE of 0.013 ± 0.001 for OLS, and 0.013 ± 0.003 for ridge, indicating higher variance in performance over folds for ridge. This occurred at degree 12 with penalty parameter $\lambda = 6.2 \cdot 10^{-7}$ for ridge.

Lasso’s maximum iterations until convergence was set to 10^5 and it failed to converge for higher condition numbers. This resulted in a higher minimum MSE of 0.020 ± 0.004 at $(p = 8, \lambda = 4.8 \cdot 10^{-5})$. As the result from ridge essentially approximates OLS with such low penalty parameter, and had higher variance across fold, the best model for this problem was determined to be OLS. We rank the models:

- 1. OLS
- 2. Ridge
- 3. Lasso

We plotted various gradient descent methods as a function of iterations to closed-form solutions of OLS, Ridge and Lasso against different penalty parameters for different initial learning rates η . We compared the results with the stochastic estimators, but concluded that for our estimation purposes, accuracy wins the cost-accuracy trade-off. The best optimizers were Adam for OLS and Ridge regression, and Proximal Gradient Descent for Lasso.

BIBLIOGRAPHY

-
- [1] C. Runge, *Zeitschrift für Mathematik und Physik* **46**, 224 (1901).
 - [2] M. Hjorth-Jensen, “Applied data analysis and machine learning: Lecture notes,” <https://github.com/CompPhysics/MachineLearning> (2026).
 - [3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. (Springer, New York, 2009).
 - [4] M. Hjort-Jensen, *Machine Learning and the Physical Sciences From Discriminative methods to generative methods and reinforcement learning* (Springer, 2026).
 - [5] P. Rebeschini, “Algorithmic foundations of learning lecture 13: The lasso estimator. proximal gradient methods,” <https://www.stats.ox.ac.uk/~rebeschini/teaching/AFoL/22/material/lecture13.pdf> (2021).

Section	Level	Notes
Abstract	0	
Introduction	0	
Theory/methods	0	
Code, implementation, tests	2	Bug identification and plotting
Results and discussion	0	
Conclusion	0	

Table VI. LLM assistance

5c)

Appendix A: Use of LLM tools

1. Tools used

Tool	Accessed
GPT 5.6	gpt.uio.no september 2026
GitHub copilot	VScode september 2026
Claude Sonnet 5.5	Cursor oktober 2026

Table V. Caption

An example of use of LLM tools that needs to be declared is experiencing complete loss of variance over bootstraps and attempting to fix the calculations myself without success. The code was shown to github copilot which identified that the same random seed was used in every bootstrap, making every partition identical causing 0 variance. Here an LLM assisted in the debugging of code, hence the level 1 assistance under code, implementations, tests.

An example of use of LLM tools that does not need to be declared is the use of an LLM to explain a concept, and then implementing text based on the newfound understanding of said concept.

An LLM (UiO GPT) was used to code the functions used for plotting figures 24, and 25.

LLM was used for finding scaling errors and other silent errors in the code. Asked to structure the code in functions in an optimized way. `jax.grad(cost)` differentiates with respect to first positional argument. The LLM was asked to rewrite the example code in the project description to compare gradients up to machine precision.

LLM was used for coding functions that visualized Figures 19, 20, 21 and 22.